

Nombre: Christhopher Irving Melgar Gómez

Grupo: 9697

Asignatura: Programación de dispositivos móviles

Actividad: 1-M03

Asesor: Cristian Cardoso Arellano

Fecha: 20-10-2026

Esta actividad es de mi autoría y las fuentes consultadas están en formato APA.

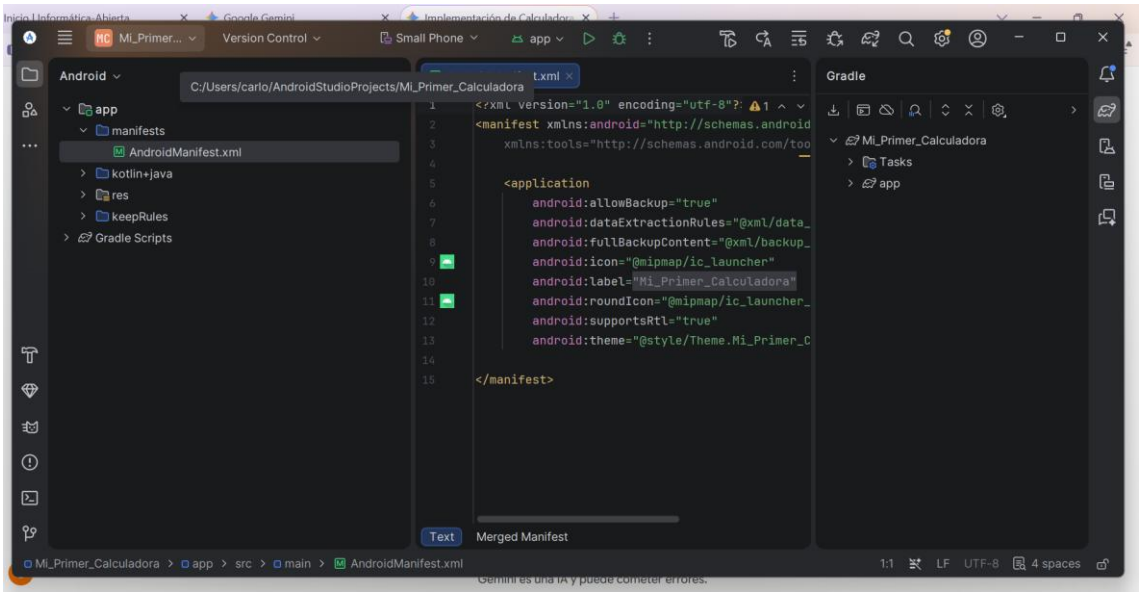
Introducción.

En la presente actividad comenzaré con la unidad 3 la cuál consiste en comprender y construir estructuras básicas de programación en Java. Para ello estaré estudiando los principales conceptos como interface, tipos de valores, operaciones, funciones matemáticas e importación de herramientas que me permitan trabajarlo para una aplicación móvil.

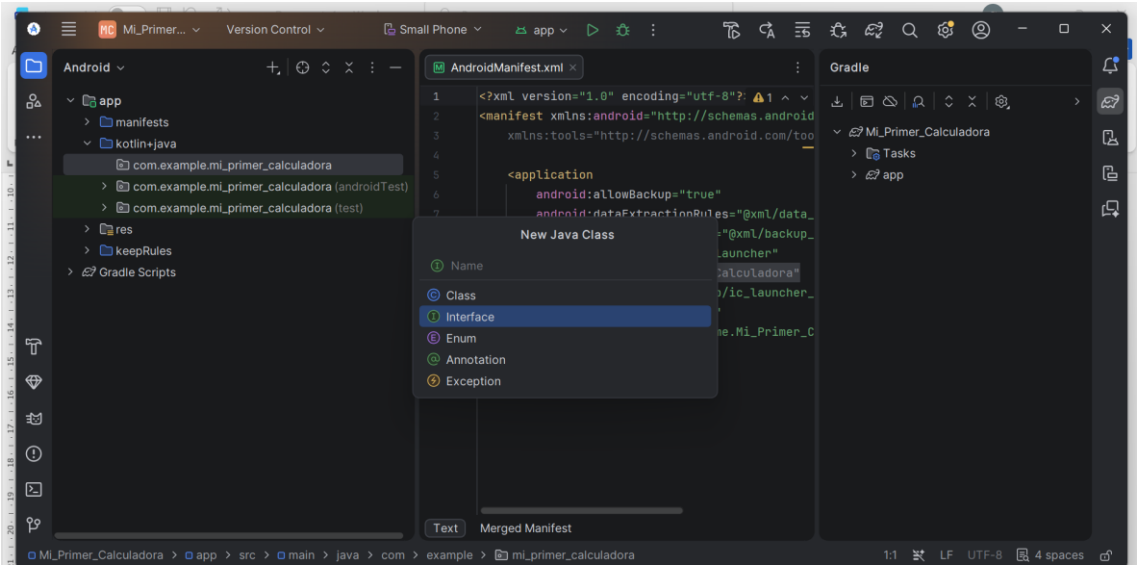
En esta ocasión de diseñará una calculadora la cuál además de realizar las operaciones aritméticas, también me permitirá determinar si una operación es válida o no mediante NaN.

Desarrollo de la actividad.

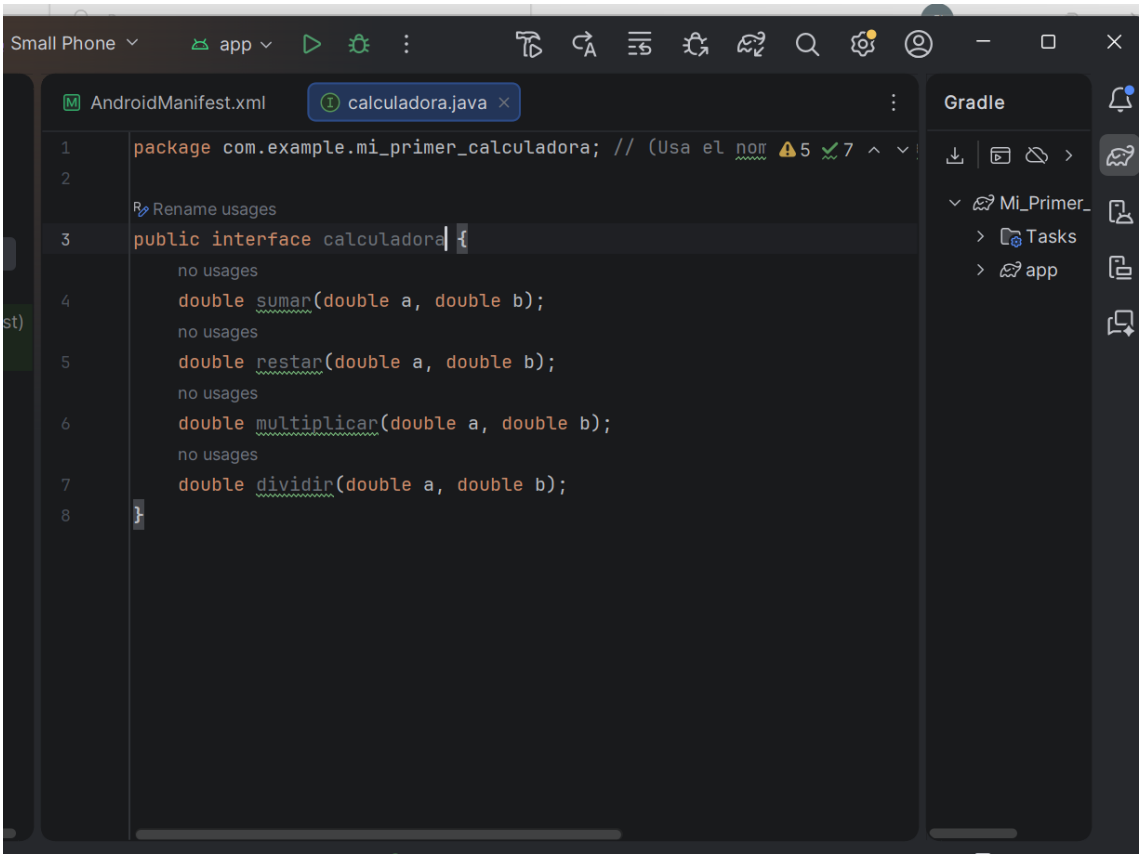
1) En principio, crearé un nuevo Proyecto bajo el nombre “Mi_primer_calculadora” En la cuál mediante el MainActivity codificaré tanto la interface como también la estructura del Código para cumplir con su función.



Esta actividad es de mi autoría y las fuentes consultadas están en formato APA.



2) El siguiente paso es codificar la interface principal, la cuál servirá para que el usuario únicamente pueda resolucionar para las operaciones implicadas. Ahora mismo doy una explicación para que son cada palabra reservada de este pedazo de Código.



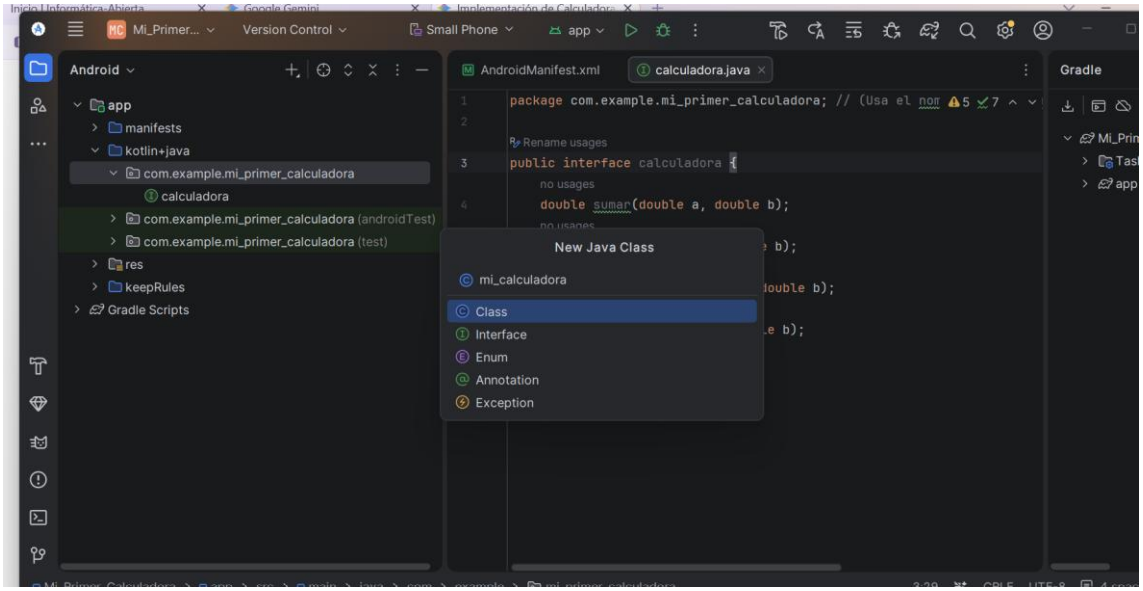
Esta actividad es de mi autoría y las fuentes consultadas están en formato APA.

Interface: Indica que acciones deberá realizar el programa, este no es aún el contenido, se trata de una Plantilla con parámetros, tipos de valor, es como el molde.

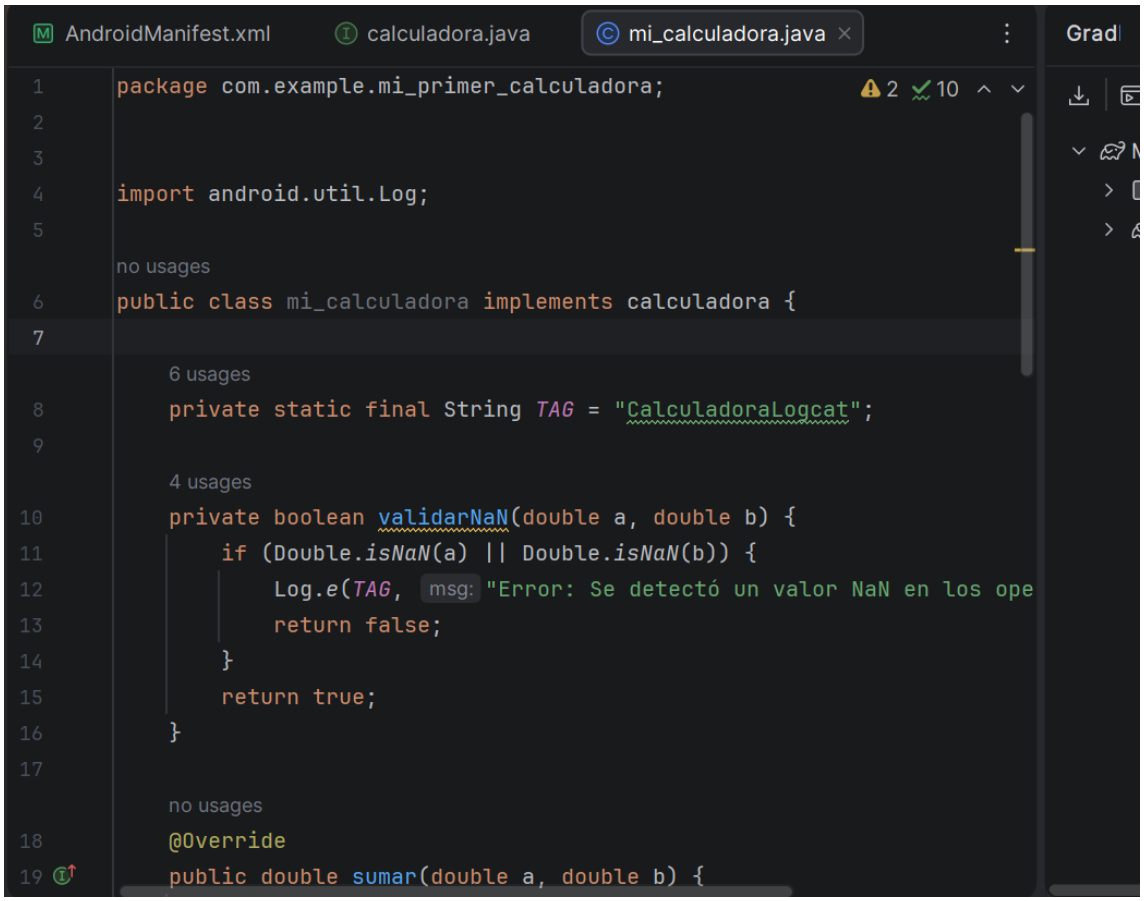
Public class: La clase la cuál vamos mandar a llama en el contenido de Nuestro programa.

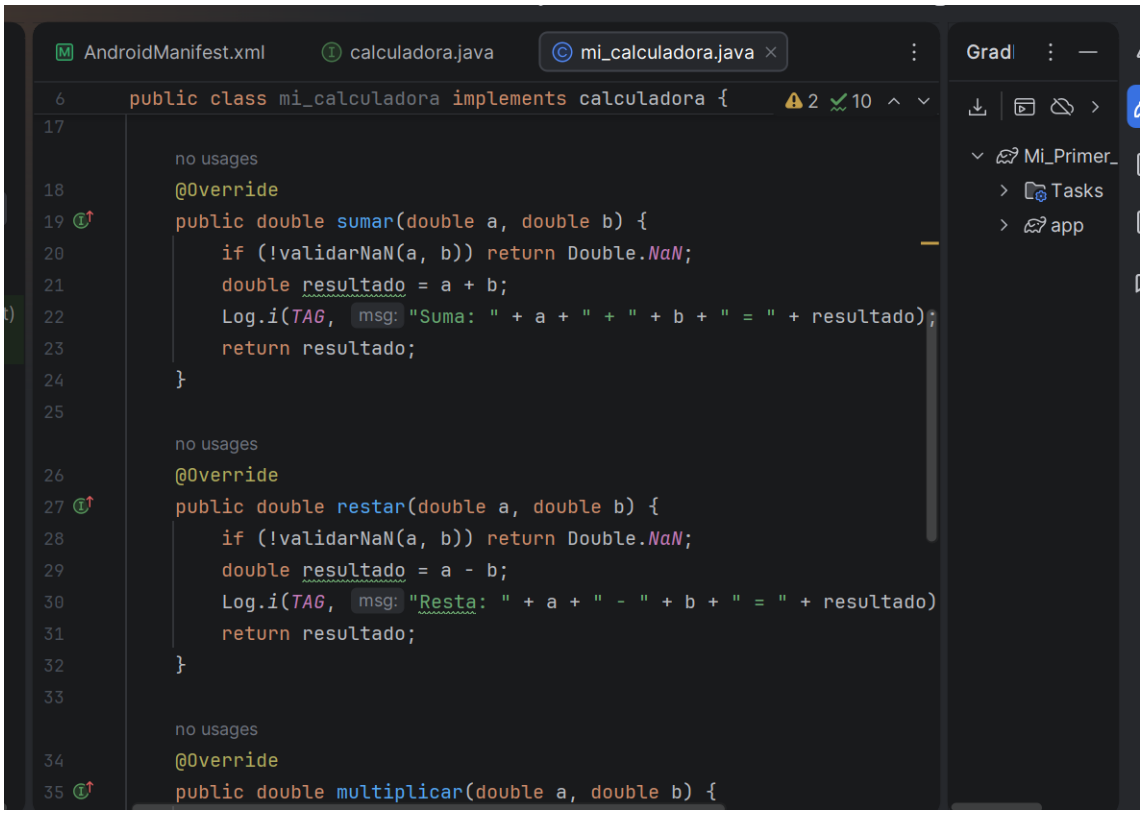
Implements: La conexión entre la clase y Plantilla, donde le deberemos describir el nombre de la interface que mandamos a llamar.

3) Lo siguiente que voy a crear es la clase Java para la calculadora, para ello me voy al Proyecto principal y creo la clase Java denominada como mi_calculadora



Esta actividad es de mi autoría y las fuentes consultadas están en formato APA.





Esta actividad es de mi autoría y las fuentes consultadas están en formato APA.

```
no usages
@Override
public double multiplicar(double a, double b) {
    if (!validarNaN(a, b)) return Double.NaN;
    double resultado = a * b;
    Log.i(TAG, msg: "Multiplicación: " + a + " * " + b + " = " + r
    return resultado;
}

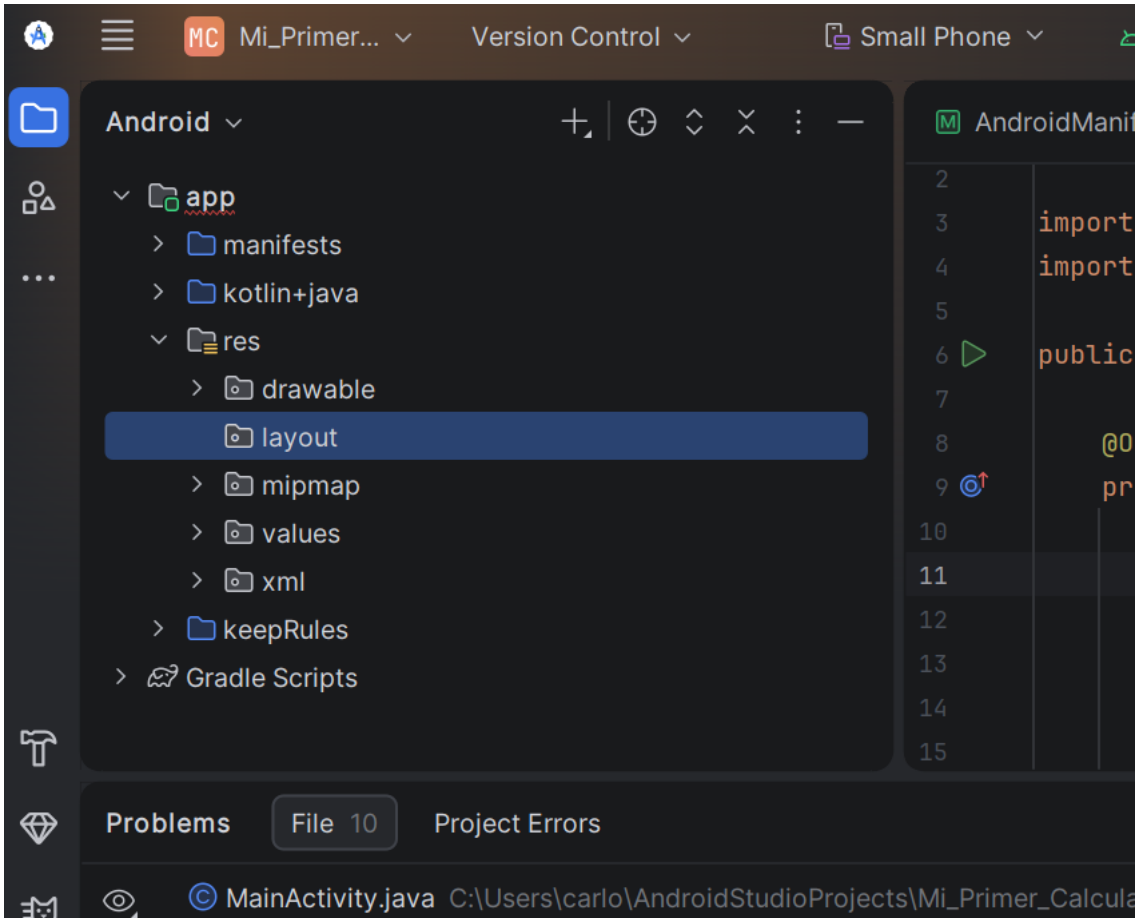
no usages
@Override
public double dividir(double a, double b) {
    if (!validarNaN(a, b)) return Double.NaN;

    if (b == 0.0) {
        Log.e(TAG, msg: "Error: División entre cero no permitida (
        return Double.NaN;
    }
```

```
41
        no usages
42      @Override
43      public double dividir(double a, double b) {
44          if (!validarNaN(a, b)) return Double.NaN;
45
46          if (b == 0.0) {
47              Log.e(TAG, msg: "Error: División entre cero no permitida (
48              return Double.NaN;
49          }
50
51          double resultado = a / b;
52          Log.i(TAG, msg: "División: " + a + " / " + b + " = " + resulta
53          return resultado;
54      }
55  }
```

Esta actividad es de mi autoría y las fuentes consultadas están en formato APA.

4) Ahora en la Main Activity es donde estaré codificando la parte de LogCat como se muestra a continuación, para ello crearé la clase MainActivity.java. Antes debo de crear el Directorio layout.

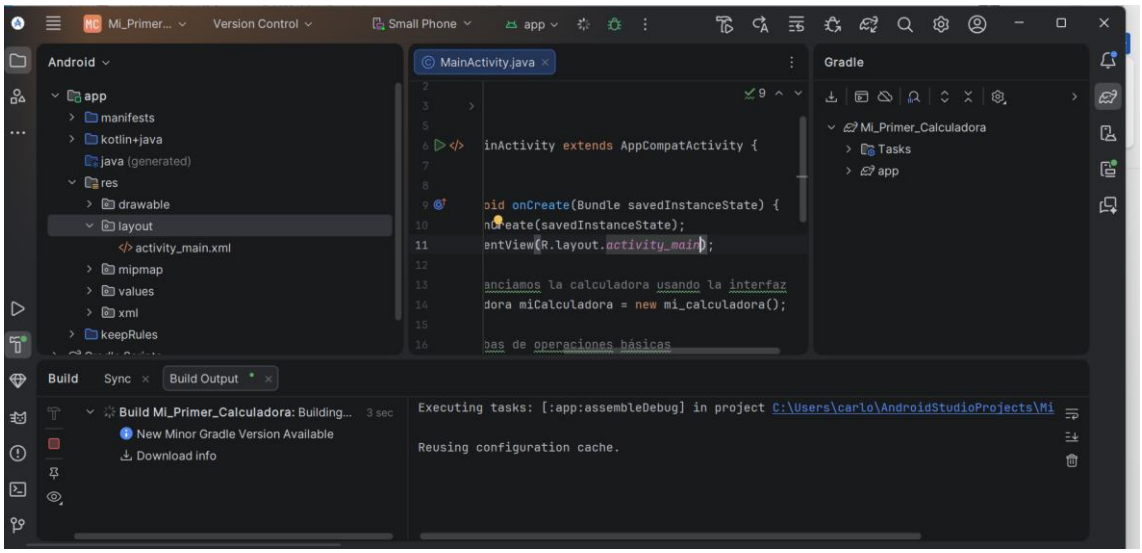


Después, voy a crear el archive xml activity_main, para el LogOut.

5) Por último, correré la aplicación para ver desde el logout su ejecución



Esta actividad es de mi autoría y las fuentes consultadas están en formato APA.



Conclusiones

1. **Uso práctico de Interfaces en Java:** Aprendí a definir una interfaz (CalculadoraInterface) para establecer la estructura obligatoria de las 4 operaciones básicas (suma, resta, multiplicación y división). Esto ayuda a separar el "qué debe hacer" del "cómo se programa", logrando un código más limpio y ordenado.

2. **Manejo e interpretación de registros con Logcat:** Al desarrollar una aplicación sin interfaz gráfica (GUI), comprendí la importancia de utilizar Logcat (android.util.Log). Usar etiquetas como Log.i para mensajes informativos y Log.e para errores permite monitorear y verificar en tiempo real que los cálculos se estén ejecutando correctamente en la consola.

3. **Validación de errores matemáticos y valores NaN:** Implementé la detección preventiva de valores no numéricos (NaN - *Not a Number*) mediante Double.isNaN(), además de controlar la división entre cero. Esto evita que el programa genere resultados indeterminados o falle durante la ejecución.

4. **Estructura de proyectos y resolución de problemas en Android Studio:** Se reforzó la correcta organización de paquetes (com.example...), la importancia de la coincidencia exacta entre el nombre de las clases Java y sus archivos, así como la resolución de errores comunes como la falta de recursos en la carpeta res/layout y la sincronización con Gradle.

Esta actividad es de mi autoría y las fuentes consultadas están en formato APA.